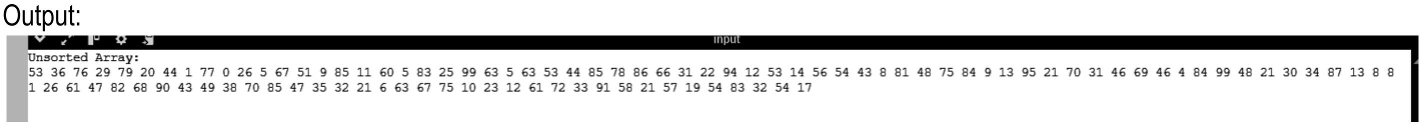
Hands-on Activity 7.1	
Sorting Algorithms	
Course Code: CPE010	**Program:** Computer Engineering
Course Title: Data Structures and Algorithms	**Date Performed:** 10/16/2024
Section: CPE21S4	**Date Submitted:** 10/18/2024
Name(s): Rio, Aries	**Instructor:** Ma'am Maria Rizette Sayo

6. Output

Code:

```
C/C++
#include <iostream>
#include <cstdlib>
#include <ctime>

int main() {
    const int arrSize = 100;
    int arr[arrSize];

    // Seed the random number generator
    srand(static_cast<unsigned int>(time(0)));

    // Generate random values for the array
    for (int i = 0; i < arrSize; i++) {
        arr[i] = rand() % 100; // Generate random values between 0 and 99
    }

    // Print the unsorted array
    std::cout << "Unsorted Array:" << std::endl;
    for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Output:



The array is made up of 100 randomly generated integers ranging from 0 to 99. The randomness of these values allows for an effective test of sorting algorithms. Since a random number generator is used, each time the program runs, it produces a different unsorted array.

Table 7-1. Array of Values for Sort Algorithm Testing

Code:

C/C++

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <algorithm>

template <typename T>
void bubbleSort(T arr[], size_t arrSize){
    for(int i = 0; i < arrSize; i++){
        for(int j = i+1; j < arrSize; j++){
            if(arr[j]>arr[i]){
                std::swap(arr[j], arr[i]);
            }
        }
    }
}

int main() {
    const int arrSize = 100;
    int arr[arrSize];

    // Seed the random number generator
    srand(static_cast<unsigned int>(time(0)));

    // Generate random values for the array
    for (int i = 0; i < arrSize; i++) {
        arr[i] = rand() % 100; // Generate random values between 0 and 99
    }

    // Print the unsorted array
    std::cout << "Unsorted Array:" << std::endl;
    for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    // Sort the array using bubble sort
    bubbleSort(arr, arrSize);

    // Print the sorted array
    std::cout << "Sorted Array:" << std::endl;
    for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
```

```
    }
    std::cout << std::endl;

    return 0;
}
```

Output:

```
Unsorted Array:
31 27 35 85 8 27 75 38 21 58 80 78 50 73 34 29 51 89 37 27 7 55 89 45 25 56 15 20 78 11 20 9 90 55 94 99 35 69 89 8 79 69 86 81 42 20 63 46 61 0 73 69 8 14 66 33 71 33 6 49 44 26 10 35 34
 4 86 69 25 75 77 56 44 63 90 38 35 53 84 96 53 10 17 13 24 84 47 47 17 53 96 14 31 6 1 65 62 87 86 39
Sorted Array:
99 96 96 94 90 90 89 89 89 87 86 86 86 85 84 84 81 80 79 78 78 77 75 75 73 73 71 69 69 69 69 66 65 63 63 62 61 58 56 56 55 55 53 53 53 51 50 49 47 47 46 45 44 44 42 39 38 38 37 35 35 35 3
5 34 34 33 33 31 31 29 27 27 27 26 25 25 24 21 20 20 20 17 17 15 14 14 13 11 10 10 9 8 8 8 7 6 6 4 1 0
```

Bubble sort goes through the list, checks two items next to each other, and swaps them if they're in the wrong order. After each full pass, the biggest unsorted item moves to its correct spot at the end. This continues until the whole list is sorted in order from smallest to largest.

Table 7-2. Bubble Sort Technique

C/C++

```
#include <iostream>
#include <cstdlib>
#include <ctime>

template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize){
    int position, j;
    T smallestElem = A[K];
    position = K;
    for(int J=K+1; J < arrSize; J++){
        if(A[J] < smallestElem){
            smallestElem = A[J];
            position = J;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], const int N){
    int POS, temp, pass=0;
    for(int i = 0; i < N; i++){
```

```cpp
            POS = Routine_Smallest(arr, i, N);
            temp = arr[i];
            arr[i] = arr[POS];
            arr[POS] = temp;
            pass++;
        }
}

int main() {
    const int arrSize = 100;
    int arr[arrSize];

    // Seed the random number generator
    srand(static_cast<unsigned int>(time(0)));

    // Generate random values for the array
    for (int i = 0; i < arrSize; i++) {
        arr[i] = rand() % 100; // Generate random values between 0 and 99
    }

    // Print the unsorted array
    std::cout << "Unsorted Array:" << std::endl;
    for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    // Sort the array using selection sort
    selectionSort(arr, arrSize);

    // Print the sorted array
    std::cout << "Sorted Array:" << std::endl;
    for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Output:

```
input
Unsorted Array:
3 84 76 91 49 81 56 55 59 7 57 38 50 1 25 39 60 88 8 91 9 43 1 90 85 18 39 80 88 80 44 43 16 20 86 65 53 94 21 12 53 30 51 3 83 28 43 44 68 51 35 77 47 88 19 84 58 58 64 98 90 60 42 7 33
28 72 86 23 45 99 28 28 2 32 11 82 27 7 50 30 94 79 77 34 51 61 93 61 78 91 52 38 85 59 71 66 83 10 41
Sorted Array:
1 1 2 3 3 7 7 7 8 9 10 11 12 16 18 19 20 21 23 25 27 28 28 28 28 30 30 32 33 34 35 38 38 39 39 41 42 43 43 43 44 44 45 47 49 50 50 51 51 51 52 53 53 55 56 57 58 58 59 59 60 60 61 61 64 65
 66 68 71 72 76 77 77 78 79 80 80 81 82 83 83 84 84 85 85 86 86 88 88 88 90 90 91 91 91 93 94 94 98 99
```

Selection sort splits the list into two sections: one sorted and the other unsorted. It repeatedly finds the smallest (or largest) element from the unsorted part and moves it to the sorted section. By the end, the entire list is arranged in ascending order, showing how the selection process works.

Table 7-3. Selection Sort Algorithm

C/C++

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

template <typename T>
void insertionSort(T arr[], const int N){
    int K = 0, J, temp;
    while(K < N){
        temp = arr[K];
        J = K-1;
        while(temp <= arr[J]){
            arr[J+1] = arr[J];
            J--;
        }
        arr[J+1] = temp;
        K++;
    }
}

int main() {
    const int arrSize = 100;
    int arr[arrSize];

    // Seed the random number generator
    srand(static_cast<unsigned int>(time(0)));

    // Generate random values for the array
    for (int i = 0; i < arrSize; i++) {
        arr[i] = rand() % 100; // Generate random values between 0 and 99
    }

    // Print the unsorted array
    std::cout << "Unsorted Array:" << std::endl;
    for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
```

```
    // Sort the array using insertion sort
    insertionSort(arr, arrSize);

    // Print the sorted array
    std::cout << "Sorted Array:" << std::endl;
    for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Output:

```
Unsorted Array:
88 87 48 84 81 62 72 41 30 85 47 49 85 50 98 98 73 13 15 76 22 8 40 41 64 16 25 66 9 70 33 49 58 81 85 39 95 57 32 25 95 79 26 80 81 24 30 54 89 97 83 12 6 75 5 22 91 30 41 52 53 74 2 11
8 87 2 55 97 34 81 92 13 59 24 47 84 54 53 25 52 36 37 10 63 95 84 7 77 77 59 30 4 13 41 12 53 43 67 50
Sorted Array:
2 4 5 6 7 8 8 9 10 11 12 12 13 13 13 15 16 22 22 24 24 25 25 25 26 30 30 30 30 32 33 34 36 37 39 40 41 41 41 41 43 47 47 48 49 49 50 50 52 52 53 53 53 54 54 55 57 58 59 59 62 63 64 66 67
70 72 73 74 75 76 77 77 79 80 81 81 81 81 83 84 84 84 85 85 85 87 87 88 89 91 92 95 95 95 97 97 98 98 100
```

Insertion sort creates the final sorted array by adding one item at a time, comparing and placing each element in its correct position. It's efficient for small datasets and performs well when the array is already partly sorted. By the end, the array is sorted in ascending order, with elements placed in the right spot as the algorithm moves along.

Table 7-4. Insertion Sort Algorithm

7. Supplementary Activity

C/C++

```
#include <iostream>
#include <cstdlib> // for rand() and srand()
#include <ctime>   // for time()

using namespace std;

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
```

```
        arr[j + 1] = key;
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    srand(time(0)); // Seed for random number generation
    int A[101];
    for (int i = 0; i < 101; i++) {
        A[i] = rand() % 5 + 1; // generate random values between 1 and 5
    }

    cout << "Original Array: ";
    printArray(A, 101);

    insertionSort(A, 101);

    cout << "Sorted Array: ";
    printArray(A, 101);

    int count[6] = {0};
    for (int i = 0; i < 101; i++) {
        count[A[i]]++;
    }

    cout << "Manual Count: ";
    for (int i = 1; i <= 5; i++) {
        cout << "Candidate " << i << ": " << count[i] << " votes ";
    }
    cout << endl;

    int max_votes = 0;
    int winning_candidate = 0;
    for (int i = 1; i <= 5; i++) {
        if (count[i] > max_votes) {
            max_votes = count[i];
            winning_candidate = i;
        }
    }
```

```
    cout << "Winning Candidate: Candidate " << winning_candidate << " with
" << max_votes << " votes" << endl;

    return 0;
}
```

```
C/C++
#ifndef SORTING_ALGORITHMS_H
#define SORTING_ALGORITHMS_H

#include <iostream>

// Bubble Sort
template <typename T>
void bubbleSort(T arr[], size_t arrSize);

// Selection Sort
template <typename T>
void selectionSort(T arr[], const int N);

// Insertion Sort
template <typename T>
void insertionSort(T arr[], const int N);

// Merge Sort
template <typename T>
void mergeSort(T arr[], const int N);

#endif // SORTING_ALGORITHMS_H
```

Outputs:

```
Original Array: 1 1 1 2 1 5 4 4 3 5 5 4 5 3 2 1 1 4 5 3 2 1 1 1 2 1 2 4 3 2 3 3 1 5 1 2 5 5 3 4 3 2 1 3 4 4 2 2 3 1 5 5
5 2 4 1 5 2 1 1 4 5 1 4 4 3 2 2 1 2 1 1 3 3 2 1 3 3 5 4 3 3 5 4 5 2 2 1 5 1 4 3 2 4 2 3 1 4 4 3 3
Sorted Array: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
Manual Count: Candidate 1: 25 votes Candidate 2: 20 votes Candidate 3: 21 votes Candidate 4: 18 votes Candidate 5: 17 vo
tes
Winning Candidate: Candidate 1 with 25 votes
```

```
Original Array: 2 3 1 2 2 5 4 2 3 5 3 2 1 3 2 1 2 5 5 2 4 4 2 2 5 2 4 1 5 1 4 3 5 5 4 4 1 1 3 3 1 2 3 2 5 4 5 4 2 4 4 4
4 4 5 3 1 2 1 4 5 4 5 5 3 5 1 5 1 1 2 2 3 5 4 4 5 4 4 3 5 5 4 3 2 3 1 4 2 4 1 3 2 3 3 4 2 5 5 5 4
Sorted Array: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
Manual Count: Candidate 1: 15 votes Candidate 2: 21 votes Candidate 3: 17 votes Candidate 4: 25 votes Candidate 5: 23 vo
tes
Winning Candidate: Candidate 4 with 25 votes
```

```
Original Array: 4 5 2 3 1 2 5 5 2 1 3 3 1 1 4 4 1 1 1 4 1 5 2 2 4 3 5 4 3 1 1 5 1 3 1 5 2 5 3 3 4 4 1 2 5 1 1 1 4 5 3 4
1 5 3 4 1 3 1 3 3 3 1 5 2 5 5 1 4 2 5 4 5 1 4 5 2 1 4 4 2 4 4 4 5 2 4 3 4 1 1 4 5 5 1 1 2 1 2 1 3
Sorted Array: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
Manual Count: Candidate 1: 29 votes Candidate 2: 14 votes Candidate 3: 16 votes Candidate 4: 22 votes Candidate 5: 20 vo
tes
Winning Candidate: Candidate 1 with 29 votes
```

```
START
  // Initialize the array with 100 vote values (1 to 5)
  Initialize array arr[100] with given values

  // Initialize an array to count votes for each candidate
  Initialize array votes[5] to {0, 0, 0, 0, 0}

  // Function to count votes
  FUNCTION votesCounter(arr, size, votes):
    FOR each element in arr:
      Increment votes[element - 1]  // Increment the corresponding index in votes

  // Function to determine the winner
  FUNCTION Winner(votes, numCandidates):
    Set maxVotes to votes[0]
    Set winner to 0

    FOR each candidate from 1 to numCandidates - 1:
      IF votes[current candidate] > maxVotes:
        Set maxVotes to votes[current candidate]
        Set winner to current candidate's index

    RETURN winner

  // Main execution
  Main:
    Initialize arr with 100 elements
    Initialize votes with 5 elements set to 0
    Call votesCounter(arr, size, votes)  // Count the votes
    Set winner to the result of Winner(votes, 5)  // Determine the winner
    PRINT votes for each candidate
    PRINT "Winner is candidate:", winner + 1  // Print the winning candidate (index + 1)
STOP
```

8. Conclusion

In this activity, I learned how to implement basic sorting algorithms, specifically insertion sort, to count votes for candidates effectively. The algorithm successfully generated an array of votes ranging from 1 to 5, sorted them, and identified the winning candidate based on the highest vote count. This hands-on experience reinforced my understanding of sorting techniques and their practical applications. Overall, I feel I did well but see room for improvement in exploring more advanced algorithms for larger datasets.

9. Assessment Rubric